Práctica 2: Pruebas de carga, Locust y Prometheus

Evaluación de Sistemas Informáticos

Grupo 16

4 de mayo de 2026

Índice

1. Descripción general del entorno
2. ¿Cómo hemos trabajado?
3. Modificaciones en el código para la práctica
4. Enfoque del monitoreo
5. Pruebas de Carga con Locust
6. Grafana

¿Qué elementos son clave?

- Servidor de autenticación OAuth2 desarrollado en Python (Flask y SQLite)
- Implementa flujos clave del RFC 6749: *Authorization Code*, *Client Credentials* y *Refresh Token*.
- Ejecutado mediante el gestor `uv` en el puerto 5001.

Distribución del trabajo

Marcos	33 %	Desarrollo de scripts del fichero Locust y redacción del cuaderno de bitácora
Edgar	33 %	Desarrollo de las metricas y de *models.py*,conexión con el Grafana y documentación
Álvaro	33 %	Gestión de la VM y del repositorio de GitHub, modificación de *oauth2_routes*, creación del *test.py* y documentación

Modificaciones en el Backend

La instrumentación se ha integrado siguiendo el principio DRY (intentando evitar los *circular imports*):

- `metrics.py`: Centralización de métricas (Counters, Gauges, Histograms) para evitar importaciones circulares. Uso estratégico de etiquetas (ej. `grant_type`)
- `models.py`: Auditoría de la base de datos (códigos generados/consumidos, detección de tokens revocados).
- `oauth2_routes.py`: Seguimiento del ciclo de vida de los flujos de autenticación (iniciados, exitosos, fallidos).

Arquitectura de Monitorización

Enfoque EDR (*Endpoint Detection and Response*):

- **Rendimiento:** Histogramas para evaluar tiempos de respuesta y control de peticiones simultáneas.
- **Estado de Tokens:** Monitorización de tokens activos, revocados, códigos pendientes y expirados.
- **Seguridad:** Detección de ataques de fuerza bruta (M2M y Web) e intentos de *Token Replay*.

Diseño de Escenarios en Locust

En el `locustfile.py` hemos intentado simular un comportamiento real"**50 % tráfico legítimo, 50 % tráfico malicioso**.

Tarea	Tipo	Peso	Prob.
`valid_client_credentials`	Tráfico legítimo M2M	4	40 %
`brute_force_api`	Ataque fuerza bruta M2M	2	20 %
`brute_force_user_login`	Ataque fuerza bruta web	2	20 %
`valid_user_login`	Tráfico legítimo web	1	10 %
`replay_attack`	Ataque token replay	1	10 %

Ejecución y Verificación

Configuración de Pruebas:

- **Carga normal:** 50 usuarios, *spawn rate* 5/s durante 5 min].
- **Prueba de estrés:** 200 usuarios, *spawn rate* 10/s durante 3 min].

Garantías del sistema:

1. Uso de script de verificación (`test_metrics.py`) para validar manualmente los 9 escenarios de métricas antes del ataque global.
2. Limitación de almacenamiento en Prometheus (`retention.size=500MB`) para proteger los recursos de la máquina virtual.

Nuestro panel de Grafana

¡Gracias por su atención!

¿Preguntas?